

LLD Notes — Phase 0 & Phase 1

Java fluency for design · Core OOP for design | Concepts, your doubts, your mistakes, interview answers

How these notes are organised. Each topic has the concept, then colour-coded boxes: **blue = a doubt you asked** (with the answer that finally made it click), **red = a mistake you actually made** (wrong → right), **green = rule to remember**, **amber = how to say it in an interview**. Re-read the red boxes before every session — they are your personal error list.

Contents

Your personal mistake summary — the habits to watch

Phase 0 — 0.1 Java vs C++ basics · 0.2 Arrays & sizes · 0.3 Collections · 0.4 int vs Integer · 0.5 Methods & classes · 0.6 Enums · 0.7 equals/hashCode · 0.8 Exceptions · 0.9 Optional · 0.10 Lambdas, ::, Comparator, Streams · 0.11 Generics

Phase 1 — 1.1 Encapsulation · 1.2 Inheritance · 1.3 Polymorphism · 1.4 Interfaces & composition · 1.5 Abstract classes · 1.6 Derived state · 1.7 Coupling, cohesion, Demeter · 1.8 Association / aggregation / composition · 1.9 Splitting a God class

Exercise log · Interview quick-recall Q&A · Pre-submit checklist

Your Personal Mistake Summary

Ranked by how often each appeared across the exercises. The top four are habits, not knowledge gaps — you knew the rule each time and slipped anyway.

#	Habit	Count	Fix
1	Using <code>[]</code> on a List (<code>students[i]</code> , <code>veh[i]</code>)	5×	<code>[]</code> works on arrays only. Lists use <code>.get(i)</code> — or better, the enhanced for-loop.
2	Accessing a private parent field from a child (<code>this.baseSalary</code> in a subclass)	4×	<code>private</code> excludes subclasses. Always use the getter: <code>getBaseSalary()</code> .
3	Compute-and-discard: building a value and never printing/returning it	4×	After each line ask: <i>does this result go anywhere?</i>
4	Missing return types (<code>public calculateMonthlyPay()</code>)	4×	Every method needs a type before its name. Only constructors have none.
5	<code>System.Out</code> / <code>system.out</code> / <code>System.println</code>	4×	Exactly <code>System.out.println</code> .
6	Passing a class name or raw value where an object is needed	3×	<code>new</code> turns a class into an object: <code>new EmailChannel()</code> , <code>new Seat("A1", GOLD)</code> , <code>throw new X()</code> .
7	<code>.length</code> on a List	2×	arrays <code>.length</code> · collections <code>.size()</code> · String <code>.length()</code>
8	Not finishing the last tasks of an exercise / dropping responsibilities	4×	Tick every requirement against your code before submitting. In timed rounds this costs marks directly.
9	<code>println</code> with two arguments (<code>println("Shortfall", x)</code>)	2×	One argument. Join with <code>+</code> .
10	Pricing logic leaking into other classes	2×	Ask "who changes this rule?" If finance/marketing, it's its own class.

Phase 0 — Java Fluency for Design

0.1 Java vs C++ — the basics

```
public class Main {                                // public class name == file name
    public static void main(String[] args) {
        int x = 10;                                // primitive
        String name = "Ram";                        // String is a class, not char[]
        System.out.println(name);
    }
}
```

- No pointers, no manual memory. Everything except primitives (`int`, `double`, `boolean`, `char`, `long` ...) is a **reference** to an object on the heap. Garbage collector frees memory — no `delete`.
- Arrays are bounds-checked: `arr[5]` on size 5 throws `ArrayIndexOutOfBoundsException` instead of silently corrupting memory like C++.
- Java is case-sensitive everywhere: `System` not `system`, `main` not `Main`. A method called `Main` means your program never starts.
- Array brackets go on the type: `String[] args`, not C-style `String args[]`.
- Java has **no truthiness**. `if (obj)`, `if (!list)`, `if (0)` are compile errors. Always compare explicitly: `obj == null`, `list.isEmpty()`, `count == 0`.

String immutability — and why

```
String s = "hello";
s = s + " world";           // creates a NEW String; the old one is unchanged

StringBuilder sb = new StringBuilder(); // use this when building in loops
sb.append("hello").append(" world");
String result = sb.toString();
```

Interview: why is String immutable? (1) **String pool** — identical literals share one object; if mutable, changing one would silently change all. (2) **Thread safety** for free. (3) **Safe as HashMap keys** — a key whose hash changes after insertion becomes unfindable. (4) **Security** — file paths, URLs, credentials can't be altered after validation. Point 3 is the one interviewers like most.

Your mistake (exercise 1): `int sum=0:` (colon) and `system.out.println` (lowercase). Loop `i` from 1 to 5 writing `arr[i]` → index 5 out of bounds. **Your fix was correct:** `arr[i-1] = i`.

0.2 Arrays, and the three "size" methods

```
int[] arr = new int[5];           // fixed size, filled with 0
int[] arr2 = {1, 2, 3, 4};
```

Type	Count	Why different
Array (<code>int[]</code> , <code>String[]</code>)	<code>.length</code> — field, no parentheses	Arrays are a built-in language feature, length is baked in
List, Set, Map	<code>.size()</code> — method	Ordinary classes, so it's a method call
String	<code>.length()</code> — method	Ordinary class

Rule: if it's behaviour, it has `()`. Fields don't. `entry.getValue` without parentheses is a compile error — it's `entry.getValue()`.

0.3 Collections

```
import java.util.*;

List<String> names = new ArrayList<>();           // ≈ C++ vector
names.add("a"); names.get(0); names.size(); names.remove(0); names.contains("a");

Map<Integer, String> users = new HashMap<>();     // ≈ unordered_map
users.put(1, "Ram"); users.get(1); users.containsKey(1); users.remove(1);

Set<String> unique = new HashSet<>();             // ≈ unordered_set, drops duplicates
unique.add("x"); unique.contains("x");

Queue<Integer> q = new LinkedList<>(); q.offer(1); q.poll(); q.peek();
Deque<Integer> stack = new ArrayDeque<>(); stack.push(1); stack.pop(); stack.peek();
```

Iterating — the part that was new

```
for (String name : names) {                                // List / Set
    System.out.println(name);
}

for (Map.Entry<Integer, String> entry : users.entrySet()) { // Map — key+value pairs
    System.out.println(entry.getKey() + " -> " + entry.getValue());
}
users.keySet();      // only keys
users.values();      // only values
```

Program to an interface. Declare `List` on the left, `ArrayList` on the right. Swap to `LinkedList` later and nothing else breaks. This is your first taste of loose coupling.

No primitives in collections. `List<int>` is illegal — use `List<Integer>`. Autoboxing converts automatically.

Your mistakes (exercises 2 & 2b):

- ✗ `for(int i=0, students.length; i++)` — comma instead of semicolon, missing condition, `.length` on a `List`
- ✗ `students[i]` — `[]` on a `List`
- ✗ "for loop logic same as previous" for the `Map` — it isn't; `Maps` need `entrySet()`
- ✗ Built the `Set` and never printed it · `cities.get(1)`; alone on a line (fetched, discarded)
- ✗ `entry.getValue` without `()` · `" has"` with no spaces → `Mumbai has20`
- ✓ Correct: `for (String city : cities), for (Map.Entry<String,Integer> entry : population.entrySet())`

0.4 int vs Integer

Your doubt: "when to use Int and int?"

Int doesn't exist in Java. There are two real things: **int** — a primitive, raw 32-bit value, no methods, can never be null, defaults to 0. **Integer** — a class wrapping an int in an object; has methods, can be null.

Use int for fields, parameters, return types, loop counters — ~90% of the time. **Use Integer** only when forced: inside collections (`List<Integer>`) or when you need null to mean "no value".

```
List<Integer> nums = new ArrayList<>();
nums.add(5);           // autoboxing: int → Integer
int x = nums.get(0);    // unboxing: Integer → int
```

Gotcha 1 — null unboxing: `int s = scores.get("missing");` → get returns null → unboxing null throws `NullPointerException`.

Gotcha 2 — == on objects: `Integer a = 1000, b = 1000; a == b` is **false** (compares references). Use `a.equals(b)`. Java caches `-128..127`, so small numbers *appear* to work with `==` — which makes the bug nastier.
Same trap with `Strings`: `ps.getSpotId() == spotId` (your exercise 10) compares references. Always `.equals()`. Only use `==` for primitives and for `== null`.

Your mistake (exercise 4): `private Int houseParked;` → `private int hoursParked;`. Also `Double total = 0;` doesn't compile — an int literal can't auto-box into `Double`. Use `double total = 0;`.

Same pairs: `double/Double`, `boolean/Boolean`, `char/Character`, `long/Long`.

0.5 Methods, classes, constructors, static, access

```
public static int add(int a, int b) { return a + b; }
// public = who can call · static = belongs to class · int = return type · add = verb name

public static double add(double a, double b) { return a + b; } // overloading:
// same name, different PARAMETERS. Return type alone can't distinguish.
```

```
public class Student {
    private String name;           // fields private by default
    private static int totalStudents = 0; // ONE copy shared by all students

    public Student(String name) { // constructor: class name, NO return type
        this.name = name;        // this.name = field, name = parameter
        totalStudents++;
    }
    public String getName() { return name; } // instance method
    public static int getTotalStudents() { return totalStudents; } // Student.getTotalStudents()

    @Override
    public String toString() { return "Student{name=" + name + "}"; }
}
Student s = new Student("Ram"); // new allocates + calls constructor
```

- **static** belongs to the class; one copy. A static method can't read instance fields — there's no object.
- If a method uses no instance fields, it probably should be static.
- Without `toString()`, printing gives `Student@1b6d3586`.

Modifier	Visible from
private	this class only (subclasses excluded)
(none)	same package
protected	same package + subclasses
public	everywhere

Rule: fields private; methods public only if they're part of what the class offers. The set of public methods *is* the class's contract. Also: private is **per-class**, **not per-object** — code inside `Seat` can read another `Seat`'s private fields (this is why `equals` works).

One public top-level class per file, and its name must match the file. In single-file exercises, make only `Main` public.

No bare statements at class level. Only fields, methods, constructors and initialiser blocks go in the class body. Your exercise 9 closed `main` after one line, leaving all the try/catch blocks floating in the class — syntax error.

Your mistakes (exercise 3 — BankAccount):

- ✗ `getBalance(){ ... }` — no return type → `public double getBalance()`
- ✗ `toString()` printed instead of returning. It must **return** a `String` so the caller decides what to do with it (log it, print it, build a report).
- ✗ `@Override()` → `@Override`
- ✗ `bA.toString();` — built and discarded. Use `System.out.println(bA)`; `println` calls `toString` automatically.
- ✗ Fields without `private` — anyone could write `bA.balance = -5000`, making validation pointless.
- ✓ Guard clauses (`if bad` → `return early`) were correct.

0.6 Enums

Replace fragile Strings (`"UPI"`, `"upi"`, typos compile fine) with a fixed, compiler-checked set.

```
public enum SeatCategory {
    SILVER(150),          // COMMAS between constants
    GOLD(250),
    PLATINUM(400);        // ONE semicolon ends the list

    private final double price;

    SeatCategory(double price) {      // implicitly private – never public
        this.price = price;
    }
    public double getPrice() { return price; }
    public double priceFor(int n) { return n * price; }
}

SeatCategory c = SeatCategory.GOLD;
c.priceFor(3);                // 750.0
```

Your doubt: "how to iterate values(), you didn't give a proper example"

`values()` returns a plain **array** of all constants, in declaration order:

```
for (SeatCategory c : SeatCategory.values()) {
    System.out.println(c + ": 3 seats = " + c.priceFor(3));
}
// SILVER: 3 seats = 450.0
// GOLD: 3 seats = 750.0
// PLATINUM: 3 seats = 1200.0

SeatCategory[] all = SeatCategory.values();    // it IS an array, so [] and .length are correct here
for (int i = 0; i < all.length; i++) System.out.println(i + ". " + all[i]);
```

- `valueOf("GOLD")` `String` → enum (throws if invalid) · `name()` → `"GOLD"` · `ordinal()` → position (don't rely on it).
- In switch, no prefix: `case BIKE -> ...; case CAR, TRUCK -> ...;`
- `values()` builds a new array each call — store it if looping repeatedly.

Enum vs class hierarchy: if types differ only in **data** (rate, spots needed) → enum. If behaviour differs in **structure** → Strategy. Your exercise-4 Bike/Truck subclasses only varied a rate — an enum would have been lighter.

Your mistakes (exercise 8): semicolons between constants; `public SeatCategory(...)` constructor (compile error); lowercase `Silver` (convention is `UPPER_CASE`); `this.price()` — price is a field, not a method.

0.7 equals() and hashCode()

By default, `equals()` inherited from `Object` is just `==` — reference identity. Two seats with identical data are "not equal". In collections this silently breaks things:

```
Set<Seat> set = new HashSet<>();
set.add(new Seat("A1", GOLD));
set.add(new Seat("A1", GOLD));
set.size(); // 2 – duplicate got in

Map<Seat, String> m = new HashMap<>();
m.put(new Seat("B2", SILVER), "Ram");
m.get(new Seat("B2", SILVER)); // null – can't find it. No exception, just wrong.
```

Your doubt: "tell me what the internal code does and what we are supposed to do"
A `HashMap` is an **array of buckets**. It wants $O(1)$ lookup, so it jumps straight to one bucket instead of scanning everything.

```
// put(key, value) – roughly:
int hash = key.hashCode(); // STEP 1: ask the object for a number
int index = hash % buckets.length; // STEP 2: number → bucket
for (Entry e : buckets[index]) // STEP 3: search ONLY that bucket, using equals()
    if (e.key.equals(key)) { e.value = value; return; }
buckets[index].add(new Entry(key, value));

// get(key) – SAME steps 1 and 2, then searches only that one bucket.
```

The failure, step by step: default `hashCode()` comes from the memory address. `s1` hashes to bucket 9 and is stored there. A new equal `s2` hashes to bucket 7 — `get` searches bucket 7, finds nothing, returns null. The data sits untouched in bucket 9.

Why overriding only `equals()` doesn't help: the wrong bucket is picked in step 2, *before* `equals()` is ever called. Your perfect equals is never reached. `hashCode()` **decides where to look**; `equals()` **decides which one it is once you're there**.

Your doubt: "hashCode is generated internally?"

What	Who
Default <code>hashCode</code> from memory address (every object has one, from <code>Object</code>)	JVM
Custom <code>hashCode</code> from your fields	You
Deciding which fields define identity	You — only you know a seat is identified by ID, not category
Calling <code>hashCode()</code> at the right moments (put/get/add/contains)	<code>HashMap</code> / <code>HashSet</code> — you never call it yourself

`String`, `Integer`, `Double` already override both correctly — that's why `Map<String, Seat>` just works. The moment *your own class* becomes a key, you must supply them.

The contract

- If `a.equals(b)` then `a.hashCode() == b.hashCode()` — **must**.
- Unequal objects may share a hash (a collision) — fine, `equals()` sorts it out. Costs speed, never correctness.
- `equals` must be reflexive, symmetric, transitive, consistent; `x.equals(null)` is false.

The template — memorise this

```
@Override
public boolean equals(Object o) {
    if (this == o) return true; // parameter MUST be Object
    if (o == null || getClass() != o.getClass()) return false; // same object
    Seat other = (Seat) o; // null / wrong type
    return Objects.equals(seatId, other.seatId); // cast – now fields are readable
    // null-safe compare of identity fields
}

@Override
public int hashCode() {
    return Objects.hash(seatId); // SAME fields as equals
}
```

The one rule: both methods use the same fields. equals on `seatId` + `hashCode` on `seatId`+category = broken. Identity fields should be `final` — if a hashed field changes while the object is in a `HashSet`, it's stranded in the old bucket and can never be found or removed.

Your mistakes (exercise 8):

- ✗ `Objects.equals(o.seatId, this.seatId)` — `o` is still typed `Object` and has no `seatId`. Cast first: `Seat other = (Seat) o;`
- ✗ `public hashCode() / @Override equals(Object o)` — missing return types `int / boolean`
- ✗ `Objects.hashCode(seatId)` — both exist! `Objects.hash(...)` is the one for building a hash from fields.
- ✗ `seats.add("A1", "Gold")` — `Set.add` takes ONE argument (you were thinking of `Map.put`), and it needs a `Seat` object: `seats.add(new Seat("A1", SeatCategory.GOLD))`
- ✗ Skipped the `Map` lookup and `contains()` — the two steps that actually prove `hashCode` works.
- ✓ The four-step equals body, comparing only `seatId` — correct from memory. That's the hard part.

Interview: "why override hashCode when you override equals?" Because hash-based collections use `hashCode` to pick the bucket before calling equals. Equal objects with different hashes land in different buckets, so lookups fail silently. Bonus: `record Seat(String id, SeatCategory c)` auto-generates both — but can't express "compare by ID only".

0.8 Exceptions

Your doubt: "what does an exception even do when thrown?"

throw is an emergency exit. The method stops instantly — no return value, nothing after it runs. `return` is walking out the front door; `throw` is jumping out the window.

It then **climbs the call chain** to the caller, and the caller's caller, looking for a `catch`. If nobody catches it, the program crashes.

```
public static void main(String[] args) {
    System.out.println("A");
    doPayment();                // exception arrives here
    System.out.println("D");    // never runs — no catch
}
static void doPayment() {
    System.out.println("B");
    account.withdraw(5000);     // throws here
    System.out.println("C");    // never runs
}
// Output: A B then crash with stack trace

// With a catch in main:
try { doPayment(); }
catch (InsufficientBalanceException e) { System.out.println("Payment failed: " + e.getMessage()); }
System.out.println("D");
// Output: A B Payment failed: ... D ("C" still skipped — catching doesn't rewind)
```

Why exceptions beat print statements

```
// print version
account.withdraw(5000);                // prints "Insufficient balance", returns normally
System.out.println("Withdrawal complete!"); // runs — a lie
shipTheOrder();                        // runs — order ships unpaid!

// throw version
account.withdraw(5000);                // throws — execution stops HERE
shipTheOrder();                        // skipped
```

A print is a message, and messages can be ignored. An exception is an interruption, and interruptions can't.

Your doubt: "what do you mean by unchecked exactly?"

"Checked" means checked by the compiler. That's all. **Checked** (extends `Exception`) — the compiler *refuses to compile* unless you catch it or declare `throws`. **Unchecked** (extends `RuntimeException`) — the compiler says nothing; if nobody catches it, crash at runtime. **Same runtime behaviour.** The only difference is whether the compiler nags you beforehand. Which one you get is decided purely by the parent class you extend.

```
Throwable
├── Error                (JVM problems — never catch)
├── Exception            ← CHECKED (IOException, SQLException)
│   └── RuntimeException  ← UNCHECKED (NullPointerException, IllegalArgumentException, IllegalStateException, yours)
```

You've met unchecked ones constantly: `NullPointerException`, `ArrayIndexOutOfBoundsException`, `NumberFormatException` — none needed a try/catch to compile.

Why business rules are usually unchecked

A checked exception forces `throws` onto every method in the chain (`checkout()` → `processPayment()` → `transfer()` → `withdraw()`), even ones that can't handle it. Signatures get polluted, and developers start swallowing it just to silence the compiler. Spring throws almost entirely unchecked exceptions.

Interview answer: "Unchecked, because business-rule violations shouldn't force throws declarations through every layer — but the compiler won't remind callers, so it needs clear documentation." Naming the tradeoff is what makes it a good answer.

Syntax

```
class InsufficientBalanceException extends RuntimeException {
    private final double shortfall;                // exceptions can carry data
    public InsufficientBalanceException(double balance, double requested) {
        super("Insufficient balance: have " + balance + ", requested " + requested); // FIRST
        this.shortfall = requested - balance;
    }
    public double getShortfall() { return shortfall; }
}

throw new InsufficientBalanceException(balance, amount);    // exceptions are OBJECTS — need new

try { ... }
catch (InsufficientBalanceException e) { e.getMessage(); e.getShortfall(); } // specific first
catch (IllegalArgumentException | InvalidAmountException e) { ... }           // multi-catch
finally { ... } // runs whether or not it failed — so its message must be true either way

try (Scanner sc = new Scanner(System.in)) { ... } // try-with-resources: auto-closes
```

super() must be the first statement. If you need computation before it, inline it inside `super(...)` or use a **private static helper** — static methods can run before the object exists: `super(buildMessage(balance, requested));`

Three rules: never swallow silently (`catch (Exception e) { }`); don't catch what you can't handle; preserve the cause when wrapping: `throw new BookingFailedException("...", e);`

Your mistakes (exercise 9):

- ✗ throw InvalidAmountException(...) → throw new ... (5 times)
- ✗ Building the message then calling super(message) second — compile error
- ✗ e.message, e.shortfall → e.getMessage(), e.getShortfall() (private fields again)
- ✗ throw new InsufficientBalanceException(balance, requested) — no variable requested in withdraw; it's amount
- ✗ finally printing "Transaction successful" — it runs after failures too
- ✓ Computing shortfall in the constructor, and catching specific before general, were correct.

Three ways to signal failure — pick deliberately

Option	Use when
Throw an exception	Failure is exceptional; the caller must react
Return Optional	Absence is a normal, expected outcome (a finder found nothing)
Return a result object	Failure is common and you need rich detail (e.g. many bookings fail because seats are taken). Exceptions are expensive and shouldn't drive normal flow.

0.9 Optional

```
// BAD — the signature lies; sometimes there's no Seat
public Seat findSeat(String id) { ...; return null; }

// GOOD — absence is part of the contract
public Optional<Seat> findSeat(String id) {
    for (Seat s : seats) if (s.getSeatId().equals(id)) return Optional.of(s);
    return Optional.empty();
}

Optional.of(x)           // x must not be null
Optional.ofNullable(x)   // null → empty
Optional.empty()

result.ifPresent(s -> System.out.println(s));
result.ifPresentOrElse(s -> ..., () -> System.out.println("none"));
result.orElse(defaultSeat);
result.orElseGet(() -> createSeat());
result.orElseThrow(() -> new SeatNotFoundException(id)); // most useful in LLD
result.map(Seat::getCategory).orElse(null);               // transform without unwrapping
```

Your doubt: "Don't use Optional for fields — what does this mean?"

Optional is a **box** — a real extra object wrapping your value. As a field, every instance carries a box forever, for nothing: inside your own class you can just check `== null`. The box earns its keep at the **boundary** — when handing a value to someone else, it tells them "this might be nothing". *You don't keep milk in a delivery carton in your own fridge; you use the carton when sending it to someone.*

```
private PaymentMethod method; // plain nullable field
public Optional<PaymentMethod> getMethod() { // wrap only when handing it out
    return Optional.ofNullable(method);
}
```

Also don't use Optional for parameters (overload instead), and never write `optional.get() != null`.

Your mistakes (exercise 10):

- ✗ return ps; in a method returning Optional → return Optional.of(ps);
- ✗ .orElseThrow(new SpotNotFoundException(...)) → needs a **lambda**: `.orElseThrow(() -> new SpotNotFoundException(...))`. The lambda defers construction so the exception is only built if the value is absent. Same reason `orElseGet(() -> ...)` exists.

0.10 Lambdas, method references (::), Comparator, Streams

Your doubt: "explain lambdas properly, don't throw jargon"

You sometimes need to **pass behaviour** as an argument — e.g. a filter whose condition varies. Java has no free-floating functions, so the old way was a whole class:

```
interface Condition { boolean test(String s); }

class IsLong implements Condition {           // old way – lots of ceremony
    @Override
    public boolean test(String s) { return s.length() > 5; }
}

filter(names, new IsLong());

filter(names, s -> s.length() > 5);           // lambda – IDENTICAL meaning
```

The lambda is the same thing with the ceremony removed. Java still creates an object implementing `Condition`.

```
public boolean test(String s) { return s.length() > 5; }
      |               |
      s               s.length() > 5
// dropped: class, method name, return type, param type, "return", braces
```

Form	Example
No parameters	<code>() -> System.out.println("hi")</code>
One parameter (parens optional)	<code>s -> s.length()</code>
Two parameters (parens required)	<code>(a, b) -> a + b</code>
Multiple statements — braces + explicit return	<code>s -> { String u = s.toUpperCase(); return u + "!"; }</code>

Where lambdas work: only where an interface with **exactly one abstract method** is expected (a "functional interface") — so Java knows which method you're implementing. Your own `PaymentMethod` qualifies: `PaymentMethod cash = amount -> System.out.println("Paid " + amount + " in cash");` For LLD, prefer named classes when the strategy carries state (card number, UPI ID); lambdas for stateless one-liners.

Your doubt: "tell me ::"

When a lambda does nothing but call one existing method, the parameter appears twice and adds nothing. `::` removes it. **Pure shorthand — no new capability.** `Employee::getName` reads as "call `getName()` on whatever comes in".

Lambda	Method reference
<code>e -> e.getName()</code>	<code>Employee::getName</code>
<code>s -> s.length()</code>	<code>String::length</code>
<code>s -> System.out.println(s)</code>	<code>System.out::println</code>
<code>s -> Integer.parseInt(s)</code>	<code>Integer::parseInt</code>
<code>() -> new ArrayList<>()</code>	<code>ArrayList::new</code>

Can't use `::` if there's extra work: `e -> e.getName().toUpperCase()` or `e -> e.getSalary() * 12` stay as lambdas.

Comparable vs Comparator

Your doubt: "how to write all these in a class — take an example"

```
class Employee implements Comparable<Employee> {    // (1) on the class line
    private String name, department;
    private double salary;
    // constructor, getters ...

    @Override
    public int compareTo(Employee other) {           // (2) a normal method inside the class
        return Double.compare(this.salary, other.salary); // neg / 0 / pos
    }

    public static final Comparator<Employee> BY_NAME = // optional: named orderings
        Comparator.comparing(Employee::getName);
    public static final Comparator<Employee> BY_DEPT_THEN_NAME =
        Comparator.comparing(Employee::getDepartment).thenComparing(Employee::getName);
}

Collections.sort(emps);                             // (3) uses compareTo – natural order
emps.sort(Comparator.comparing(Employee::getName)); // (4) Comparator written AT THE CALL SITE
emps.sort(Employee.BY_DEPT_THEN_NAME);
```

	Where it lives	How many
Comparable	inside the class, as <code>compareTo</code>	one natural order per class
Comparator	outside, where you sort (or as a static field)	as many as you want

Decoding the Comparator lines

```
Comparator.comparing(Employee::getName)
// = (a, b) -> a.getName().compareTo(b.getName()) - you say WHAT to extract, Java writes the compare

Comparator.comparingDouble(Employee::getBaseSalary) // comparingInt/Long/Double avoid boxing
Comparator.comparing(Employee::getBaseSalary).reversed() // build it, then flip: highest first

Comparator.comparing(Employee::getDepartment)
    .thenComparing(Employee::getName)
// sort by department; ONLY when two are equal on department, break the tie by name

// All three identical:
emps.sort((a, b) -> a.getName().compareTo(b.getName()));
emps.sort(Comparator.comparing(e -> e.getName()));
emps.sort(Comparator.comparing(Employee::getName));
```

Rules: use `Integer.compare(a, b)` / `Double.compare` — never subtraction (overflows). `compareTo` is the instance method (`a.compareTo(b)`); `compare` is the static one (`Integer.compare(a, b)`). In an interview, a working raw lambda beats a half-remembered method reference.

Your doubt: "can't we do `return this.spots.sort(...)`?"

No — two reasons. (1) `List.sort()` returns **void**; it sorts in place and hands back nothing, so there's nothing to return. (2) Even if it compiled, it would reorder the list's own internal list as a side effect of a *query*. **Queries must not mutate state.**

Method	Returns	Effect
<code>list.sort(cmp)</code> / <code>Collections.sort(list)</code>	<code>void</code>	mutates in place
<code>stream().sorted(cmp)</code>	new <code>Stream</code>	original untouched

```
// Option A: copy, then sort
List<ParkingSpot> copy = new ArrayList<>(spots);
copy.sort(Comparator.comparingInt(ParkingSpot::getFloor)
    .thenComparingInt(ParkingSpot::getDistanceToGate));
return copy;

// Option B: stream (one statement)
return spots.stream()
    .sorted(Comparator.comparingInt(ParkingSpot::getFloor)
        .thenComparingInt(ParkingSpot::getDistanceToGate))
    .collect(Collectors.toList()); // .toList() on Java 16+ gives an IMMUTABLE list
```

Streams — enough to read and write

```
emps.stream().filter(e -> e.getBaseSalary() > 40000).collect(Collectors.toList());
emps.stream().mapToDouble(Employee::getBaseSalary).sum();
emps.stream().collect(Collectors.groupingBy(Employee::getDepartment)); // Map<String, List<Employee>>

// Nearest free spot — min() returns Optional automatically (filtering may leave nothing)
spots.stream()
    .filter(s -> !s.isOccupied())
    .min(Comparator.comparingInt(ParkingSpot::getDistanceToGate));
```

Functional interface	Shape	Use
<code>Predicate<T></code>	<code>T → boolean</code>	filtering
<code>Function<T,R></code>	<code>T → R</code>	transforming
<code>Consumer<T></code>	<code>T → void</code>	side effects
<code>Supplier<T></code>	<code>() → T</code>	lazy creation (<code>orElseThrow</code> , <code>orElseGet</code>)
<code>Comparator<T></code>	<code>(T,T) → int</code>	ordering

Your mistakes (exercise 10):

- ✗ `class ParkingSpot extends Comparable` → implements (it's an interface)
- ✗ `INT.compareTo(a, b) → Integer.compare(a, b)`
- ✗ `this.spots.filter(...)[0]` — `List` has no filter (need `.stream()`), and `[]` on a `List` again
- ✗ `Collections.sort(this.spots)` inside a finder — mutates state
- ✗ `this.spots = [];` → `new ArrayList<>()` · `this.isOccupied = false` — field is `occupied`, `isOccupied()` is the getter
- ✗ Ternary without parentheses in `toString`: `"..." + occupied ? "free" : "taken"` — `?:` binds looser than `+`. Write `(occupied ? "occupied" : "free")`.

0.11 Generics

You've used them all along (`List<String>`). They move type errors from runtime to compile time and remove casts.

```
public class Repository<T> { // T = "some type, decided by the user"
    private final Map<String, T> store = new HashMap<>();
    public void save(String id, T item) { store.put(id, item); }
    public Optional<T> findById(String id) { return Optional.ofNullable(store.get(id)); }
    public List<T> findAll() { return new ArrayList<>(store.values()); }
}
Repository<ParkingSpot> repo = new Repository<>(); // one class, any entity

public static <T> void printAll(List<T> items) { ... } // generic method: <T> before return type

class PriceCalculator<T extends Payable> { ... } // bounded: T must implement Payable,
// so item.getPrice() is allowed
```

Interview extras: Type erasure — generics exist only at compile time; at runtime `List<String>` is just `List`, so no `new T()`, no `T[]`, no `instanceof List<String>`. **Wildcards** — `List<? extends Vehicle>` = readable list of some Vehicle subtype; recognition level is enough.

Phase 1 — Core OOP for Design

1.1 Encapsulation

The object controls access to its own data. Fields private; state changes only through methods that enforce the rules.

Why it matters: if `balance` is public, anyone can set it negative and your `withdraw()` checks become decorative. Validation only means something when there's no way around it. Interviewers scan your field declarations early.

Related: state changes through intention-revealing methods — `book.markIssued()`, `spot.occupy()` — not by callers setting fields.

1.2 Inheritance

```
public class SavingsAccount extends BankAccount {
    private double interestRate;

    public SavingsAccount(String acc, String holder, double rate) {
        super(acc, holder);           // parent constructor – MUST be first
        this.interestRate = rate;
    }
    public void addInterest() {
        deposit(getBalance() * interestRate / 100);    // getter + method, NOT this.balance
    }
}

@Override                          // always write it – catches typos in the method name
public void withdraw(double amount) { ... }           // replace parent behaviour

@Override
public void deposit(double amount) {
    super.deposit(amount);           // extend parent behaviour
    System.out.println("logged");
}
```

private excludes subclasses. A child inherits the *methods* to read state, not direct access to private fields. Going through `deposit()` is better anyway — validation still runs. If you find yourself wanting `protected` fields so children can poke state directly, that's often a sign the child shouldn't be a child.

```
public final class Constants { }      // can't be extended
public final void validate() { }      // can't be overridden
private final String accountNumber;  // can't be reassigned after construction – great for IDs
```

Your mistakes (exercise 4 — Vehicle/Bike/Truck):

- ✗ `return 10 * this.hoursParked;` in `Bike` — private in `Vehicle` → `10 * getHoursParked()`
- ✗ `@override` → `@Override` (annotations are case-sensitive)
- ✗ `regNumber()` getter → `getRegNumber()` — frameworks like Spring rely on the `getX` convention
- ✗ field `houseParked` vs usage `hoursParked` — two unrelated names to the compiler
- ✗ `for (i<3) veh[i]` — hardcoded bound + `[]` on List; `return total;` inside void `main`; nothing printed
- ✓ `Vehicle bike1 = new Bike(...)` — polymorphic declaration, unprompted. `super()` first.

1.3 Polymorphism

```
List<BankAccount> accounts = new ArrayList<>();
accounts.add(new SavingsAccount(...));
accounts.add(new CurrentAccount(...));
for (BankAccount acc : accounts) {
    acc.withdraw(5000);    // Java calls the ACTUAL object's version at runtime
}
```

The loop is typed as the general type; each object supplies its own behaviour. This **runtime dispatch** is the mechanism every design pattern is built on. Also visible: a parent's `toString()` calling `calculateFee()` shows the Truck fee for a Truck.

	Overloading	Overriding
What	same name, different parameters	same signature in a subclass
Resolved	compile time	runtime
Where	same class (usually)	parent → child

Upcasting (`BankAccount a = new SavingsAccount();`) is always safe and implicit.

1.4 Interfaces and composition (favour composition over inheritance)

The problem inheritance creates: weekend rates × monthly passes × EV fees × vehicle types = `WeekendMonthlyPassTruck` ... a combinatorial class explosion, all fixed at compile time. And a `FixedDepositAccount` that refuses withdrawals breaks the promise its parent made (that's a Liskov violation — Phase 2).

```
interface FeeStrategy { double calculateFee(int hours); }    // pure contract

class HourlyFee implements FeeStrategy {
    private final double rate;
    public HourlyFee(double rate) { this.rate = rate; }
    @Override public double calculateFee(int hours) { return rate * hours; }
}

class Vehicle {
    private FeeStrategy feeStrategy;                // HAS-A – composition
    public Vehicle(String reg, int hours, FeeStrategy fs) { ...; this.feeStrategy = fs; }
    public double calculateFee() { return feeStrategy.calculateFee(hoursParked); } // delegate
    public void setFeeStrategy(FeeStrategy fs) { this.feeStrategy = fs; } // swap at RUNTIME
}

Vehicle truck = new Vehicle("MH03", 7, new FlatPlusHourlyFee(50, 100));
truck.setFeeStrategy(new HourlyFee(30));           // weekend rate – impossible with inheritance
```

You wrote the Strategy pattern (twice — notifications, payments) before it was named: pull the varying behaviour into an interface, hold an instance, delegate.

- Interface methods are implicitly `public abstract`. A class can implement **many** interfaces but extend only **one** class.
- Can't instantiate an interface, but can type a variable as one.

Inheritance		Composition
Bound	compile time, permanent	runtime, swappable
Coupling	tight — child depends on parent internals (fragile base class)	loose — depends only on the interface
Limit	one parent	many collaborators
Testing	hard	pass in a fake

Say it out loud: "A Truck *is a* Vehicle" — true, inheritance defensible for the type. "A Truck *is a* fee calculation" — nonsense; it *has* one. Use inheritance only for a genuine is-a where the child can substitute for the parent without surprising anyone.

Your mistakes (exercise 5 — notifications):

- ✗ `new NotificationService(EmailChannel)` — passing a class name. A class is a blueprint; `new EmailChannel()` is a thing.
- ✗ `channel.send(String recipient, String message)` — types belong in declarations, never calls → `send(recipient, message)`
- ✗ `this.NotificationChannel = channel` — capital N is the type, not your field
- ✗ `System.println` → `System.out.println` · multiple public classes in one file
- ✓ The design was fully correct: field typed as interface, constructor injection, delegation, runtime swap, zero if/else.

Your mistakes (exercise 6 — payments): nearly clean — every exercise-5 mistake fixed.

- ✗ `if (!this.paymentMethod)` — no truthiness → `if (paymentMethod == null)`
- ✗ Copy-paste: UPI class printed "via Credit card" · "Paid" + amount → `Paid300.0` (no space)
- ✗ Constructors without `public`

Error messages should name the entity: "No payment method selected for order 123", not "Error".

1.5 Abstract classes

```
public abstract class Employee {           // cannot be instantiated: new Employee() won't compile
    private String name, id;
    private double baseSalary;             // REAL shared state – an interface can't hold this

    public Employee(String name, String id, double baseSalary) { ... }
    public double getBaseSalary() { return baseSalary; }

    public abstract double calculateMonthlyPay();    // no body – every concrete child MUST implement

    public String payslip() {                 // concrete method calling the abstract one
        return name + " (" + id + "): " + calculateMonthlyPay();
    }
}
```

Fixes two exercise-4 problems: nobody can create a meaningless bare `Vehicle`, and a new subclass that forgets to implement the fee fails at compile time instead of silently charging a default. `payslip()` — parent defines the skeleton, child fills one step — is the **Template Method** pattern.

	Interface	Abstract class
Fields	only public static final constants	real instance fields
Constructor	none	yes
Method bodies	default / static only (Java 8+)	freely
How many	implement many	extend one
Relationship	"can-do" (capability)	"is-a" (family)

Interview: "interface or abstract class?" Interface for a **capability** unrelated types may share (`Comparable`, `Bonusable`). Abstract class for a **family sharing real state and code**. Default to interface — extends spends your only inheritance slot. Often both: abstract class `Vehicle` implements `Parkable`, `Billable`. Follow-up "why do abstract classes still exist after default methods?" → **state**: interfaces still can't hold instance fields or constructors.

instanceof — when it's legitimate

```
// FINE – asking "what can it do?" (checking a capability interface)
if (emp instanceof Bonusable b) {           // Java 16+: check + cast in one step
    System.out.println("Bonus: " + b.calculateBonus());
}

// SMELL – asking "what exactly is it?" and putting the logic in the caller
if (emp instanceof FullTimeEmployee) bonus = 0.2 * emp.getBaseSalary();
else if (emp instanceof Intern) bonus = 5000;    // new type = edit this if/else (Open/Closed violation)
```

To remove instanceof entirely: a Null Object — `Employee.calculateBonus()` returns 0 by default. Tradeoff: no branching, but every employee now claims a bonus capability, muddying the model.

1.6 Don't store derived state

Your mistake (exercise 7 — payroll):

```
private double bonus;                       // x derived data stored as a field
public double calculateBonus() { this.bonus = 0.2 * baseSalary; return bonus; }
public double calculateMonthlyPay() { return baseSalary + this.bonus; }    // 0 unless calculateBonus ran first!
```

- **Temporal coupling** — one method is only correct if another was called first. It doesn't crash; it returns something plausible and wrong. One of the nastiest bug types.
- **Staleness** — raise the salary and the stored bonus is out of date.
- Also conflated two concepts: monthly pay and bonus are different things. That separation is *why* `Bonusable` exists.

Rule: if a value can be computed from existing fields, compute it on demand — `return 0.2 * getBaseSalary();` — and store only independent state. (Exception: deliberate caching of something expensive, where you manage invalidation on purpose. Computing `shortfall` once in an exception's constructor is fine — its inputs can never change.)

Other exercise 7 mistakes: `super()` after field assignments in `Contractor`; `Contractor` took a `baseSalary` parameter and silently ignored it (a lying parameter — drop it); `getSalary()` declared `String` but returned `double`; `payslip` concatenation missing `+`; `System.out.printl("bonus", x)`.
Smell worth naming: passing 0 as `baseSalary` means `Employee` assumes every employee has a salary. Cleaner: move `baseSalary` down into the subclasses that actually have one.
✓ `Contractor` correctly did NOT implement `Bonusable`; `instanceof` aimed at the capability.

1.7 Coupling, cohesion, Law of Demeter

Cohesion = how focused one class is. **Coupling** = how much classes depend on each other. Target: **high cohesion, loose coupling**.

```
// LOW cohesion – a "God class"
class ParkingManager {
    void assignSpot() {...} double calculateFee() {...} void printReceipt() {...}
    void sendEmail() {...} void saveToDatabase() {...} void generateMonthlyReport() {...}
}

// TIGHT coupling
class Order { private CreditCardPayment p = new CreditCardPayment("1234"); }

// LOOSE coupling – your exercise 6
class Order { private PaymentMethod p; public Order(PaymentMethod p) { this.p = p; } }
```

Cohesion test: describe the class in one sentence. If you need "and", cohesion is low.
Tight-coupling signs: new ConcreteClass() inside business logic · reaching into another object's internals · one change forcing edits in several classes.

Law of Demeter — "only talk to your immediate friends":

```
order.getCustomer().getAddress().getCity().getPincode();    // x depends on 4 classes
order.getDeliveryPincode();                                // ✓ chain moves inside the owner

customer.getMembership().getTier().getName().equals("GOLD") // x (BookingManager)
customer.isGoldMember()                                     // ✓
```

The chain doesn't vanish — it moves into the class that owns those objects. More than one dot is worth a second look (fluent builders and streams excepted).

1.8 Association, aggregation, composition

Relationship	Meaning	Example	UML
Association	"uses" — knows about it, no ownership	Driver drives a Car (passed in); BookingService receives a Customer per call	plain line
Aggregation	"has-a", independent lifecycle	ParkingLot has Vehicles; Team has Players; a service holds injected collaborators	hollow diamond
Composition	"owns", dependent lifecycle — usually created by the whole	ParkingLot creates Floors; Order owns OrderLineItems; Ride owns its RouteSegments	filled diamond

The test: if the container is destroyed, does the part still make sense? Yes → aggregation. No → composition.

Multiplicity: 1, 0..1, 1..*, *. "A Lot has 1..* Floors; a Floor has 1..* Spots; a Spot has 0..1 Vehicle" — that last one is a real constraint your code must enforce.

Say it like this: "Floor is composition — it can't exist outside the lot, so the lot creates it." That's a precise statement about lifecycle and deletion, not just a box on a diagram.

1.9 Splitting a God class — the method

1. **List every responsibility** in the original. For each, ask "who would ask for this to change?" — different answer = different class.
2. **Turn each into an interface** with swappable implementations (Postgres/InMemory, SMS/Email, Flat/Tiered).
3. **Split multi-axis names** — `generateMonthlyReportPdf` hides period AND format → generator + formatter.
4. **Create the orchestrator** — a service with collaborators injected in the constructor and short methods that only delegate. It *coordinates*, it doesn't *do*.
5. **Fix Demeter chains** by moving them into the owning class.
6. **Replace prints with exceptions**; return values instead of printing receipts.
7. **Tick off every original method against an owner.** (Your recurring miss.)

Your key miss (exercise 11 — BookingManager): "create separate **functions** for high cohesion". **Cohesion is judged at the class level.** Six methods doing six unrelated jobs in one class is still low cohesion — tidier, not focused. And you injected dependencies as `bookSeat(...)` parameters: long-lived collaborators go in the **constructor**; method parameters are for per-call data.

✓ Correctly named: tight coupling via `new EmailSender()`, and the Demeter chain.

Missed: hardcoded if/else pricing (`Open/Closed`), `new MySQLDatabase()` (concrete vendor), payment as a print with a raw card number, receipt printing in the domain, SQL built by string concatenation (injection risk), "Already booked" as a print instead of an exception.

Reference refactor — BookingService

```
interface SeatRepository      { boolean isBooked(String seatId); void markBooked(String seatId); }
interface PricingStrategy    { double basePrice(String seatId); }
interface DiscountPolicy     { double apply(double price, Customer c); }
interface PaymentGateway     { void charge(String card, double amount); }
interface NotificationService { void notify(Customer c, String msg); }

class BookingService {
    private final SeatRepository seatRepo;      // aggregation: injected, independent lifecycle
    private final PricingStrategy pricing;
    private final DiscountPolicy discount;
    private final PaymentGateway payment;
    private final NotificationService notifier;

    public BookingService(SeatRepository r, PricingStrategy p, DiscountPolicy d,
                          PaymentGateway pay, NotificationService n) { ... }

    public double bookSeat(String seatId, Customer customer, String card) { // Customer: association
        if (seatRepo.isBooked(seatId)) throw new SeatAlreadyBookedException(seatId);
        double price = discount.apply(pricing.basePrice(seatId), customer);
        payment.charge(card, price);
        seatRepo.markBooked(seatId);
        notifier.notify(customer, "Booked " + seatId + " for " + price);
        return price; // caller decides how to show a receipt
    }
}
```

Reference answer — LibraryManager

Original method	Owner
issueBook	LendingService — orchestrator (validate member → check book → record Loan → notify)
calculateLateFine	FineCalculationStrategy — standalone (FlatPerDay, Tiered)
sendOverdueReminderSms	NotificationService — SMS is one implementation
saveToPostgres	BookRepository — Postgres is one implementation; plus Member/Loan repositories
generateMonthlyReportPdf	ReportGenerator (period → data) + ReportFormatter (PDF, Word)
validateMemberCard	MemberValidator

Your answer vs reference: right on repository, notification, fine strategy, reports. Two fixes: **finest must not depend on reports** — dependency pointed backwards (consumers depend on producers, never the reverse; a formatting change could alter how much people are charged). And **issueBook isn't member management** — lending gets its own orchestrator.

Why the reference scores well — every future change is "add one class" or "edit one place": Postgres → Mongo (one repository), SMS → WhatsApp (one notifier), student fine rule (one strategy), yearly Excel report (one formatter).

Scope in a 45-min round: name the classes, sketch the interfaces, fully implement only the core flow (`issueBook`), talk through the rest.

Drill results — three God classes

Problem	Correct	Wrong / missing
Food delivery	PaymentGateway, NotificationService, repository per entity, CouponStrategy, placeOrder in orchestrator	calculateTotal is pricing — separate PricingService. assignNearestDeliveryPartner dropped → DeliveryAssignmentStrategy. Relationship: Order→OrderItem composition, Order→DeliveryPartner association.
Ride booking	NotificationService, PaymentGateway, FareStrategy	surgeMultiplier is pricing , not matching. requestRide is the orchestrator (RideService), not matching. rateDriver dropped → RatingService. Demeter chain <code>rider.getAccount().getWallet()</code> not addressed. Ride→RouteSegment composition; Ride→Driver association.
Hotel booking	Room availability, BookingService using it, RefundCalculationStrategy separate from pricing (best call in the set) , NotificationService, third-party sync strategy (that's the Adapter pattern)	priceForStay dropped → PricingStrategy. Invoice has two axes: InvoiceGenerator + InvoiceFormatter.

The two patterns from the drill: (1) **you drop responsibilities** — go back through the original methods and tick each against an owner; (2) **pricing leaks** into other classes — whenever a number is computed, ask "who changes this rule?"

Exercise Log

#	Exercise	Concept	Main lesson
1	Array sum	syntax, bounds	<code>arr[i-1]; ;</code> not <code>: ;</code> <code>System</code> capitalised
2	Students / roll map / set	collections	no <code>[]</code> on List; <code>.size()</code> ; <code>entrySet()</code> ; print what you compute
2b	Cities / population	iteration retry	<code>getValue()</code> needs <code>()</code> ; spacing in concatenation
3	BankAccount	classes, encapsulation	return types; <code>toString</code> returns; private fields; guard clauses ✓
4	Vehicle / Bike / Truck	inheritance, polymorphism	private parent fields; <code>int</code> not <code>Int</code> ; <code>@Override</code>
5	Notification channels	interfaces, composition	<code>new</code> = object; no types at call site
6	Payment methods	Strategy, null state	no truthiness (<code>== null</code>)
7	Payroll	abstract + interface	don't store derived state; <code>super()</code> first; <code>instanceof</code> on capability
8	Seat / SeatCategory	enums, equals/hashCode	enum commas; cast before comparing; <code>Objects.hash</code>
9	BankAccount v2	exceptions	<code>throw new</code> ; getters on exceptions; code inside main
10	ParkingLot registry	Optional, Comparator, streams	<code>implements</code> ; <code>.equals()</code> on Strings; lambdas in <code>orElseThrow</code> ; queries don't mutate
11	BookingManager refactor	coupling, cohesion	split into classes , not methods; constructor injection
12	Library + 3 God classes	responsibility splitting	orchestrator; tick every method; pricing is its own class

Interview Quick-Recall Q&A

Cover the right column and answer out loud. If you can't produce the answer in one or two sentences, re-read that section.

Question	Answer in one breath
Why is String immutable?	String pool sharing, thread safety, safe HashMap keys (hash can't change), security of validated values.
String vs StringBuilder?	String creates a new object per modification; StringBuilder mutates in place — use it when building in a loop.
int vs Integer?	Primitive vs wrapper object. Use int by default; Integer for collections or when null must mean "no value". Never compare Integers with ==.
== vs equals()?	== compares references for objects (values for primitives); equals compares logical value. Use == only for primitives and null checks.
Why override hashCode with equals?	Hash collections pick the bucket with hashCode before calling equals; equal objects with different hashes are never found.
The equals/hashCode contract?	Equal objects must have equal hashes; unequal objects may collide. Both must use the same fields.
Overloading vs overriding?	Overloading: same name, different params, compile time. Overriding: same signature in subclass, runtime.
What does static mean?	Belongs to the class, one shared copy; static methods can't access instance fields.
Can a subclass access a private parent field?	No — private excludes subclasses. Use the getter or a protected accessor.
Interface vs abstract class?	Interface = capability shared by unrelated types, multiple allowed. Abstract class = family sharing state and code, one only. Default to interface.
Why do abstract classes exist after default methods?	State — interfaces can't have instance fields or constructors.
Composition vs inheritance?	Inheritance is compile-time, tight, one parent. Composition is runtime-swappable, loosely coupled, testable. Favour composition unless it's a true substitutable is-a.
What is polymorphism?	Calling a method through a general type and getting the actual object's implementation at runtime.
Checked vs unchecked exception?	Checked: compiler forces catch or throws. Unchecked (RuntimeException): no enforcement. Same runtime behaviour.
Which for business rules and why?	Unchecked — avoids throws pollution across layers; document it since the compiler won't.
What does throw actually do?	Exits the method immediately and climbs the call stack to the nearest matching catch; crashes if none.
What is finally for?	Code that must run whether or not an exception occurred — cleanup.
When to return Optional?	Return types where absence is a normal outcome. Not for fields or parameters.
orElse vs orElseGet?	orElse always evaluates its argument; orElseGet takes a lambda and evaluates only if empty.
Comparable vs Comparator?	Comparable: one natural order inside the class (compareTo). Comparator: any number of external orderings.
What is a lambda?	A short implementation of a single-method interface, without writing a class.
What is a functional interface?	An interface with exactly one abstract method — the only place a lambda can go.
What does :: mean?	Method reference — shorthand for a lambda that only calls one method.
Type erasure?	Generic types exist only at compile time; at runtime List<String> is just List.
Why enums over Strings?	Compiler-checked fixed set, can carry fields and behaviour, no typos, cleanest thread-safe Singleton.
Coupling vs cohesion?	Cohesion: how focused a class is (want high). Coupling: how dependent classes are (want loose).
Law of Demeter?	Only talk to immediate collaborators; avoid a.getB().getC().do() chains.
Aggregation vs composition?	Both has-a. Aggregation: part survives the whole. Composition: part dies with the whole and is usually created by it.
"How would you add a new payment type?"	Write one new class implementing PaymentMethod; nothing else changes.
"How would you unit test this?"	Inject fakes through the constructor — possible only because the service depends on interfaces.
When is instanceof acceptable?	Checking a capability interface ("can it do X?"), not branching on concrete type ("what is it?").
Should you store a value computable from other fields?	No — causes temporal coupling and staleness. Compute on demand unless deliberately caching.

Pre-Submit Checklist

Run through this before sending any exercise — and in an interview, before saying "done".

Syntax sweep

- ☐ Every method has a return type (void if nothing)
- ☐ System.out.println spelled exactly; one argument, joined with + , with spaces inside the quotes
- ☐ No [] on a List · .size() on collections · .equals() for Strings/objects
- ☐ Every object created with new — including throw new
- ☐ super(...) is the first line of every child constructor
- ☐ @Override on every override, capital O, no parentheses
- ☐ No if (obj) — explicit == null / .isEmpty()
- ☐ All code inside methods; only Main is public in a single file

Design sweep

- ☐ Fields private (and `final` where they never change, especially IDs); constructors and API methods public
- ☐ Subclasses use getters, never parent private fields
- ☐ Dependencies typed as interfaces and injected in the constructor — no `new ConcreteThing()` in business logic
- ☐ Every class passes the one-sentence, no-"and" test
- ☐ No derived values stored as fields
- ☐ Failures thrown as domain exceptions, not printed
- ☐ Query methods don't mutate state (copy before sorting)
- ☐ No `if/else` or `instanceof` on concrete types where polymorphism should do the work
- ☐ No `a.getB().getC()` chains

Completeness sweep

- ☐ Every requirement in the prompt ticked against a line of code
- ☐ Every computed value is printed, returned, or stored
- ☐ Every original responsibility has an owner (for refactors)
- ☐ Relationship types named where asked (association / aggregation / composition)

Status: Phase 0 ✓ **Phase 1** ✓ → **Next: Phase 2, SOLID.** Most of it will feel familiar — Single Responsibility is what you did in exercise 11, Open/Closed is why Strategy works, Dependency Inversion is constructor injection. Next session opens with a cold recall of these notes.